

ShopEase Order Service: Containerized E-Commerce Backend

Building, Deploying & Managing a Multi-Container Application with Podman

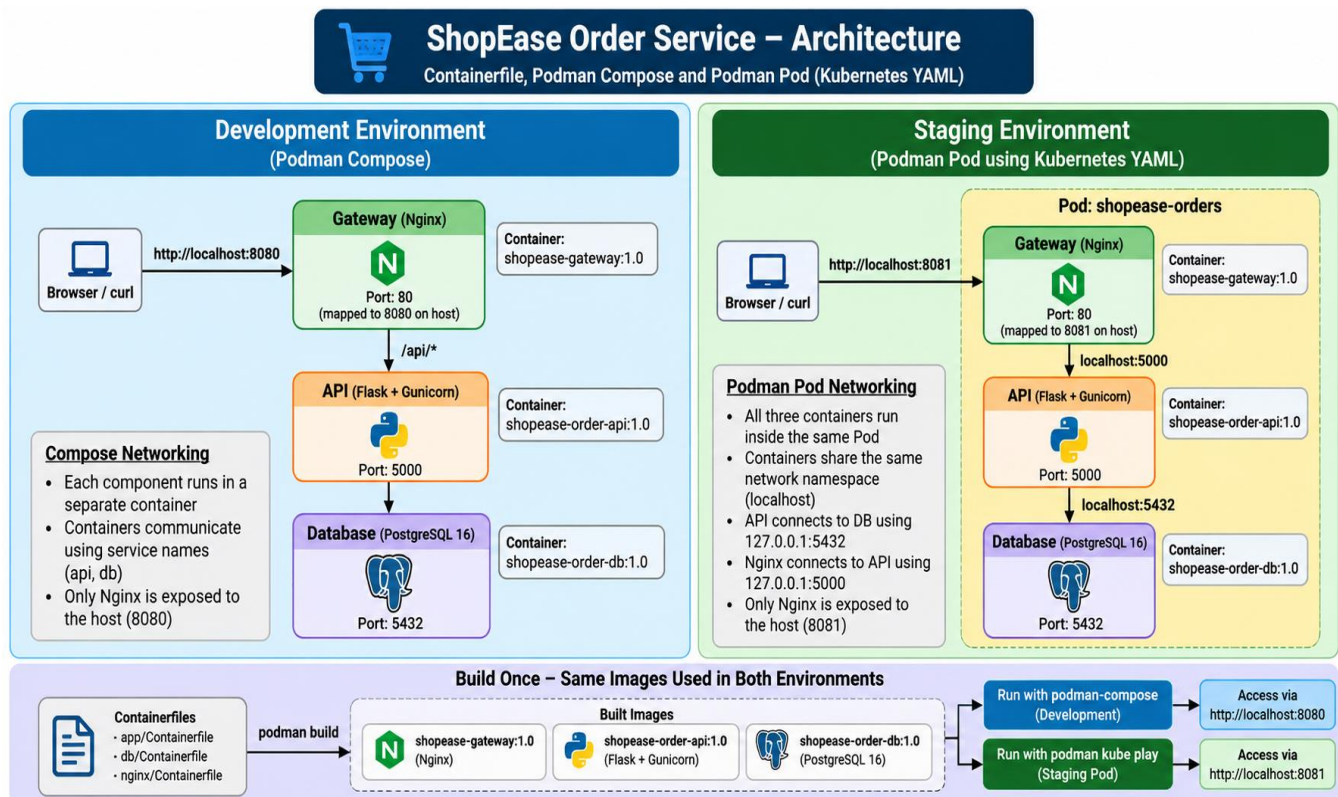
Objective:

The objective of this project is to build and deploy a simple e-commerce order service using Podman. The application consists of a Nginx gateway, a Flask API, and a PostgreSQL database. The project demonstrates how to build containers using Containerfiles, run the application locally with Podman Compose, and deploy the same container images as a Pod using Kubernetes YAML. It also shows how the containers communicate and how only the gateway is exposed to users.

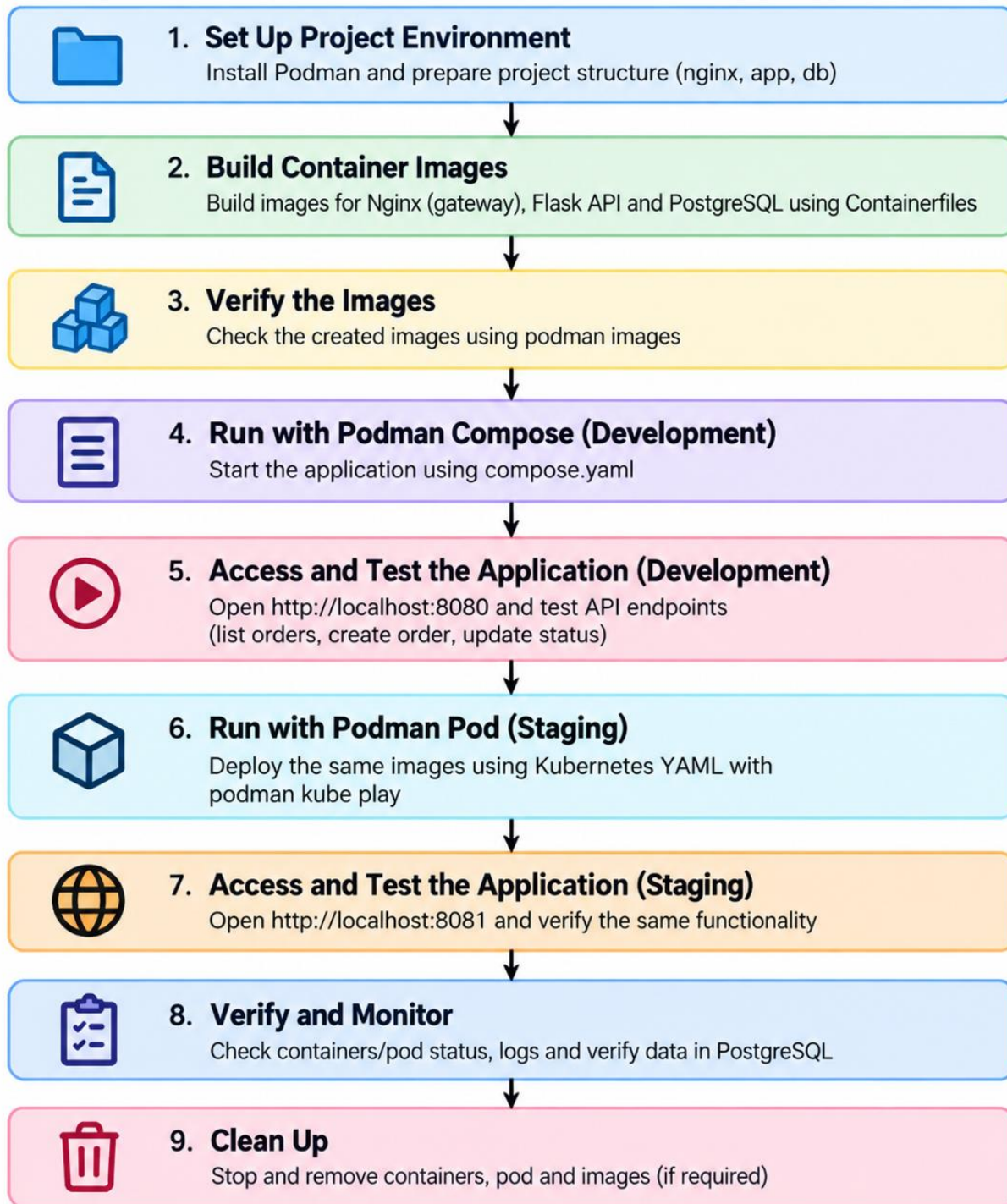
Architecture:

The ShopEase Order Service consists of three main components: Nginx, Flask/Gunicorn API, and PostgreSQL. Nginx acts as the only public entry point and forwards API requests to the Flask application, which communicates with PostgreSQL to store and retrieve order data.

The application is deployed in two environments using the same container images. During development, Podman Compose runs the services separately and they communicate using service names. For staging, the same images are deployed together in a Podman Pod using Kubernetes YAML, where the containers share the same network and communicate through 127.0.0.1.



Project Workflow:



Implementation:

Step 1: Check Environment Prerequisites

First, check that Podman and Podman-Compose are installed on the workstation.

```
podman --version
```

podman-compose --version

```
sai@ubuntu: ~  
sai@ubuntu:~$ podman --version  
podman version 4.9.3  
sai@ubuntu:~$ podman-compose --version  
podman-compose version: 1.0.6  
['podman', '--version', '']  
using podman version: 4.9.3  
podman-compose version 1.0.6  
podman --version  
podman version 4.9.3  
exit code: 0  
sai@ubuntu:~$ |
```

Step 2: Initialize Microservice Directory Structure

Create the root project folder shopease-orders and subdirectories for each tier.

mkdir -p shopease-orders

cd shopease-orders

pwd

```
sai@ubuntu: ~/shopease-ord  
sai@ubuntu:~$ mkdir -p shopease-orders  
sai@ubuntu:~$ cd shopease-orders  
sai@ubuntu:~/shopease-orders$ pwd  
/home/sai/shopease-orders  
sai@ubuntu:~/shopease-orders$ |
```

mkdir app db nginx kube

```
sai@ubuntu: ~/shopease-ord  
sai@ubuntu:~/shopease-orders$ mkdir app db nginx kube  
sai@ubuntu:~/shopease-orders$ ls  
app db kube nginx  
sai@ubuntu:~/shopease-orders$ |
```

Step 3: Develop API Microservice & Security Containerfile

Navigate to app/ directory and initialize the Python Flask application files:

cd app

touch app.py requirements.txt Containerfile

```
sai@ubuntu: ~/shopease-ord  
sai@ubuntu:~/shopease-orders$ cd app  
sai@ubuntu:~/shopease-orders/app$ pwd  
/home/sai/shopease-orders/app  
sai@ubuntu:~/shopease-orders/app$ touch app.py requirements.txt Containerfile  
sai@ubuntu:~/shopease-orders/app$ ls  
app.py Containerfile requirements.txt  
sai@ubuntu:~/shopease-orders/app$ |
```

Specify dependencies in app/requirements.txt:

flask==3.0.3

gunicorn==22.0.0

psycopg2-binary==2.9.9

```
sai@ubuntu: ~/shopease-ordl × + v
sai@ubuntu:~/shopease-orders/app$ vim requirements.txt |
```

```
sai@ubuntu: ~/shopease-ordl × + v
flask==3.0.3
gunicorn==22.0.0
psycopg2-binary==2.9.9
~
~
~
```

Implement Flask REST API in app/app.py:

```
sai@ubuntu: ~/shopease-ordl × + v
sai@ubuntu:~/shopease-orders/app$ vim app.py |
```

```
sai@ubuntu: ~/shopease-ordl × + v
import os

import psycopg2
import psycopg2.extras
from flask import Flask, jsonify, request

app = Flask(__name__)

# PostgreSQL configuration
DB_CONFIG = {
    "host": os.getenv("DB_HOST", "127.0.0.1"),
    "port": int(os.getenv("DB_PORT", "5432")),
    "dbname": os.getenv("DB_NAME", "orders"),
    "user": os.getenv("DB_USER", "shopease"),
    "password": os.getenv("DB_PASSWORD", "shopease123"),
}

def get_conn():
    return psycopg2.connect(**DB_CONFIG)

# -----
# Health Check
# -----

@app.get("/api/health")
def health():
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")

        return jsonify(
            status="ok",
            database="connected"
        ), 200
```

Create the Containerfile to build the Flask API image, run it as a non-root user (UID 1001), and use Gunicorn to serve the application.

```
sai@ubuntu: ~/shopease-ordl × + v
sai@ubuntu:~/shopease-orders/app$ vim Containerfile |
```

```

FROM docker.io/library/python:3.12-slim

LABEL maintainer="platform-team@shopease.example" \
      app="shopease-order-api" \
      version="1.0"

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN useradd --uid 1001 --no-create-home appuser

USER 1001

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
~
~

```

Step 4: Develop Database Microservice & Schema Initializer

Navigate to db/ directory and configure PostgreSQL schema and container build files:

cd db

```

sai@ubuntu: ~/shopease-ord |
sai@ubuntu:~/shopease-orders/app$ cd ..
sai@ubuntu:~/shopease-orders$ ls
app db kube nginx
sai@ubuntu:~/shopease-orders$ cd db
sai@ubuntu:~/shopease-orders/db$ pwd
/home/sai/shopease-orders/db
sai@ubuntu:~/shopease-orders/db$ |

```

Create the database schema and sample orders in db/init.sql, then prepare the PostgreSQL Containerfile.

```

sai@ubuntu: ~/shopease-ord |
sai@ubuntu:~/shopease-orders/db$ touch init.sql Containerfile
sai@ubuntu:~/shopease-orders/db$ ls
Containerfile init.sql
sai@ubuntu:~/shopease-orders/db$ |

```

```

sai@ubuntu: ~/shopease-ord |
sai@ubuntu:~/shopease-orders/db$ vim init.sql |

```

```

sai@ubuntu: ~/shopease-ord |
CREATE TABLE IF NOT EXISTS orders (
  id SERIAL PRIMARY KEY,
  customer VARCHAR(100) NOT NULL,
  item VARCHAR(100) NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  status VARCHAR(20) NOT NULL DEFAULT 'PLACED',
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

INSERT INTO orders (customer, item, quantity, status) VALUES
('Asha Rao', 'Wireless Mouse', 2, 'SHIPPED'),
('Vikram Shah', 'Mechanical Keyboard', 1, 'PACKED'),
('Neha Kulkarni', 'USB-C Hub', 3, 'PLACED');
~
~
~

```

Create the PostgreSQL Containerfile and copy init.sql into the database initialization directory so it runs when the database starts.

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders/db$ vim Containerfile

FROM docker.io/library/postgres:16-alpine
LABEL app="shopease-order-db" version="1.0"
COPY init.sql /docker-entrypoint-initdb.d/01-init.sql
EXPOSE 5432
~
~
```

Step 5: Develop Nginx Gateway Microservice & Dynamic Proxy Template

Navigate to nginx/ directory and create gateway configuration template:

cd nginx

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders/db$ cd ..
sai@ubuntu:~/shopease-orders$ ls
app db kube nginx
sai@ubuntu:~/shopease-orders$ cd nginx
sai@ubuntu:~/shopease-orders/nginx$ pwd
/home/sai/shopease-orders/nginx
sai@ubuntu:~/shopease-orders/nginx$ |
```

touch default.conf.template Containerfile

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders/nginx$ touch default.conf.template Containerfile
sai@ubuntu:~/shopease-orders/nginx$ ls
Containerfile default.conf.template
sai@ubuntu:~/shopease-orders/nginx$ |
```

Create the Nginx configuration template to route API requests using \${API_HOST}.

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders/nginx$ vim default.conf.template |
```

```
sai@ubuntu: ~/shopease-ord | X + v
upstream order_api {
    server ${API_HOST}:5000;
}

server {
    listen 80;
    server_name _;

    location / {
        default_type text/plain;
        return 200 "ShopEase Order Service - try /api/orders\n";
    }

    location /api/ {
        proxy_pass http://order_api;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 30s;
    }
}
~
~
```

Construct nginx/Containerfile leveraging Alpine Nginx base image:

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders/nginx$ vim Containerfile |

FROM docker.io/library/nginx:1.27-alpine

LABEL app="shopease-gateway" version="1.0"

COPY default.conf.template /etc/nginx/templates/default.conf.template

EXPOSE 80
~
~
~
```

Step 6: Build Container Images & Conduct Security Audit

Build container images locally using Podman and verify build artifacts:

```
cd /home/sai/shopease-orders
```

```
podman build -t localhost/shopease-order-db:1.0 ./db
```

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders$ podman build -t localhost/shopease-order-db:1.0 ./db
STEP 1/4: FROM docker.io/library/postgres:16-alpine
Trying to pull docker.io/library/postgres:16-alpine...
Getting image source signatures
Copying blob 1b7f9cd581ee done |
Copying blob e2de96513ba9 done |
Copying blob 210b280f9556 done |
Copying blob 81717818649a done |
Copying blob 3ae6141cfd99 done |
Copying blob 25b77f59fd95 done |
Copying blob 6c25da34e03f done |
Copying blob a88782f92806 done |
Copying blob d8d21be4bf02 done |
Copying blob 6117bd39f23f done |
Copying blob 509688edf37b done |
Copying config 81bd698b45 done |
Writing manifest to image destination
STEP 2/4: LABEL app="shopease-order-db" version="1.0"
--> 7e1bc9cd36e8
STEP 3/4: COPY init.sql /docker-entrypoint-initdb.d/01-init.sql
--> 241e51d7b92f
STEP 4/4: EXPOSE 5432
COMMIT localhost/shopease-order-db:1.0
--> f8be3406a36f
Successfully tagged localhost/shopease-order-db:1.0
f8be3406a36fa30144bd559db8105559bd46dba91b350832758a8d48d88e685c
sai@ubuntu:~/shopease-orders$ |
```

```
podman build -t localhost/shopease-order-api:1.0 ./app
```

```
sai@ubuntu: ~/shopease-ord | X + v
sai@ubuntu:~/shopease-orders$ podman build -t localhost/shopease-order-api:1.0 ./app
```

```
podman build -t localhost/shopease-gateway:1.0 ./nginx
```

```
sai@ubuntu: ~/shopease-ordr X + v
sai@ubuntu:~/shopease-orders$ podman build -t localhost/shopease-gateway:1.0 ./nginx
STEP 1/4: FROM docker.io/library/nginx:1.27-alpine
Trying to pull docker.io/library/nginx:1.27-alpine...
WARN[0011] Failed, retrying in 2s ... (1/3). Error: initializing source docker://nginx:1.27-alpine: ping container registry registry-1.docker.io: Get "https://registry-1.docker.io/v2/": dial tcp: lookup registry-1.docker.io on 127.0.0.53:53: read udp 127.0.0.1:50599->127.0.0.53:53: i/o timeout
Getting image source signatures
Copying blob 197eb75867ef done |
Copying blob f18232174bc9 done |
Copying blob 61ca4f733c80 done |
Copying blob b464cfd2a63 done |
Copying blob d7e507024086 done |
Copying blob 81bd8ed7ec67 done |
Copying blob 34a64644b756 done |
Copying blob 39c2ddfd6010 done |
Copying config 6769dc3a70 done |
Writing manifest to image destination
STEP 2/4: LABEL app="shopease-gateway" version="1.0"
--> 8c2caa2d4d23
STEP 3/4: COPY default.conf.template /etc/nginx/templates/default.conf.template
--> f9c012129839
STEP 4/4: EXPOSE 80
COMMIT localhost/shopease-gateway:1.0
--> 9ed872b5e198
Successfully tagged localhost/shopease-gateway:1.0
9ed872b5e19810db099861f4d903ef10feaaa8eca4be6d442ca8477b00228878
sai@ubuntu:~/shopease-orders$
```

podman images | grep shopease

```
sai@ubuntu: ~/shopease-ordr X + v
sai@ubuntu:~/shopease-orders$ podman images | grep shopease
localhost/shopease-gateway 1.0 9ed872b5e198 47 seconds ago 49.7 MB
localhost/shopease-order-api 1.0 91d0cebea430 About a minute ago 155 MB
localhost/shopease-order-db 1.0 f8be3406a36f 3 minutes ago 297 MB
sai@ubuntu:~/shopease-orders$
```

podman image inspect localhost/shopease-order-api:1.0 --format '{{.Config.User}}'

```
sai@ubuntu: ~/shopease-ordr X + v
sai@ubuntu:~/shopease-orders$ podman image inspect localhost/shopease-order-api:1.0 --format '{{.Config.User}}'
1001
sai@ubuntu:~/shopease-orders$
```

Step 7: Configure & Deploy Local Development Stack

Create compose.yaml in project root to orchestrate local development using service DNS:

```
sai@ubuntu: ~/shopease-ordr X + v
sai@ubuntu:~/shopease-orders$ pwd
/home/sai/shopease-orders
sai@ubuntu:~/shopease-orders$ vim compose.yaml
```

```
sai@ubuntu: ~/shopease-ordr X + v
services:
  db:
    build: ./db
    image: localhost/shopease-order-db:1.0
    container_name: shopease-db
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: shopease
      POSTGRES_PASSWORD: shopease123

  api:
    build: ./app
    image: localhost/shopease-order-api:1.0
    container_name: shopease-api
    environment:
      DB_HOST: db
      DB_NAME: orders
      DB_USER: shopease
      DB_PASSWORD: shopease123
    depends_on:
      - db
    restart: on-failure

  gateway:
    build: ./nginx
    image: localhost/shopease-gateway:1.0
    container_name: shopease-gateway
    environment:
      API_HOST: api
    ports:
      - "8080:80"
    depends_on:
      - api
```

Launch development stack with podman-compose and verify container status:

```
podman-compose up -d --build
```

```
sai@ubuntu: ~/shopease-ord: X + v
sai@ubuntu:~/shopease-orders$ podman-compose up -d --build
```

```
podman-compose ps
```

```
sai@ubuntu:~/shopease-ord: X + v
sai@ubuntu:~/shopease-orders$ podman-compose ps
podman-compose version: 1.0.6
['podman', '--version', '']
using podman version: 4.9.3
podman ps -a --filter label=io.podman.compose.project=shopease-orders
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
29e74a4e9a6e   localhost/shopease-order-db:1.0     postgres                34 seconds ago Up 33 seconds                shopease-db
cc848b60b1eb   localhost/shopease-order-api:1.0    gunicorn --bind 0...    30 seconds ago Up 29 seconds                shopease-api
28544cc4c78c   localhost/shopease-gateway:1.0     nginx -g daemon o...    25 seconds ago Up 24 seconds    0.0.0.0:8080->80/tcp    shopease-gateway
exit code: 0
sai@ubuntu:~/shopease-orders$ podman-compose logs api
podman-compose version: 1.0.6
['podman', '--version', '']
using podman version: 4.9.3
podman logs shopease-api
[2026-10-02 06:30:34 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-10-02 06:30:34 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-10-02 06:30:34 +0000] [1] [INFO] Using worker: sync
[2026-10-02 06:30:34 +0000] [2] [INFO] Booting worker with pid: 2
[2026-10-02 06:30:34 +0000] [3] [INFO] Booting worker with pid: 3
exit code: 0
sai@ubuntu:~/shopease-orders$
```

Step 8: End-to-End Development Testing (Port 8080)

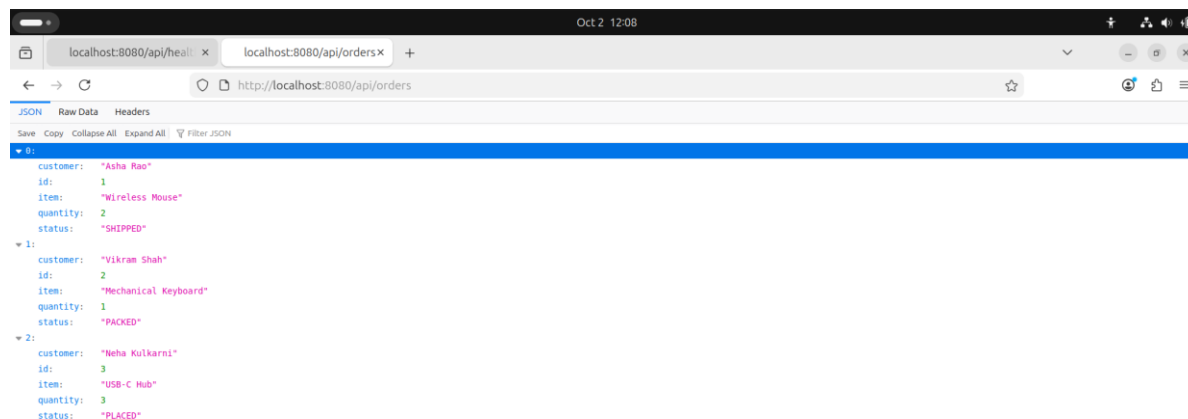
Perform end-to-end REST API testing against development gateway on <http://localhost:8080>:

```
curl http://localhost:8080/api/health
```



```
localhost:8080/api/health x +
http://localhost:8080/api/health
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
{
  "database": "connected",
  "status": "ok"
}
```

```
curl http://localhost:8080/api/orders
```

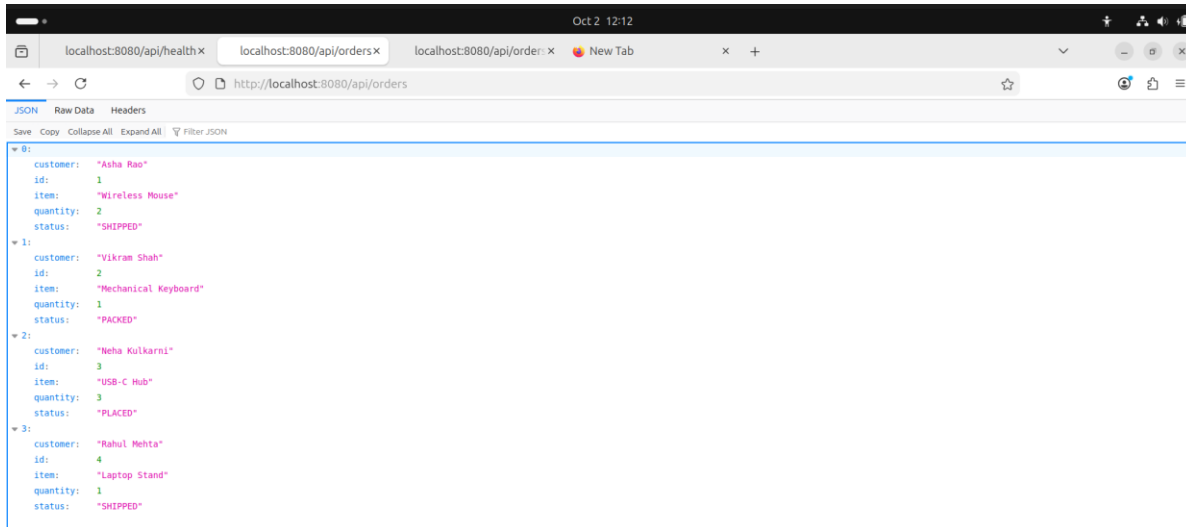


```
localhost:8080/api/health x localhost:8080/api/orders x +
http://localhost:8080/api/orders
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
[
  {
    "customer": "Asha Rao",
    "id": 1,
    "item": "Wireless Mouse",
    "quantity": 2,
    "status": "SHIPPED"
  },
  {
    "customer": "Vikram Shah",
    "id": 2,
    "item": "Mechanical Keyboard",
    "quantity": 1,
    "status": "PACKED"
  },
  {
    "customer": "Neha Kulkarni",
    "id": 3,
    "item": "USB-C Hub",
    "quantity": 3,
    "status": "PLACED"
  }
]
```

```
curl -X POST http://localhost:8080/api/orders -H "Content-Type: application/json" -d
{"customer":"Rahul Mehta","item":"Laptop Stand","quantity":1,"status":"PLACED"}
```

```
curl -X PATCH http://localhost:8080/api/orders/4 -H "Content-Type: application/json" -d
{"status":"SHIPPED"}
```

```
sai@ubuntu: ~/shopease-orders$ curl -X POST http://localhost:8080/api/orders -H "Content-Type: application/json" -d '{"customer": "Rahul Mehta", "item": "Laptop Stand", "quantity": 1}'
{"customer": "Rahul Mehta", "id": 4, "item": "Laptop Stand", "quantity": 1, "status": "PLACED"}
sai@ubuntu: ~/shopease-orders$ curl -X PATCH http://localhost:8080/api/orders/4 -H "Content-Type: application/json" -d '{"status": "SHIPPED"}'
{"customer": "Rahul Mehta", "id": 4, "item": "Laptop Stand", "quantity": 1, "status": "SHIPPED"}
sai@ubuntu: ~/shopease-orders$
```



Step 9: Security Port Audit & Development Stack Teardown

Audit host port exposures using podman port before tearing down development stack:

podman port shopease-gateway

podman port shopease-api

podman-compose down

```
sai@ubuntu: ~/shopease-orders$ podman port shopease-gateway
80/tcp -> 0.0.0.0:8080
sai@ubuntu: ~/shopease-orders$ podman port shopease-api
sai@ubuntu: ~/shopease-orders$ podman-compose down
podman-compose version: 1.0.6
['podman', '--version', '']
using podman version: 4.9.3
** excluding: set()
podman stop -t 10 shopease-gateway
shopease-gateway
exit code: 0
podman stop -t 10 shopease-api
shopease-api
exit code: 0
podman stop -t 10 shopease-db
shopease-db
exit code: 0
podman rm shopease-gateway
shopease-gateway
exit code: 0
podman rm shopease-api
shopease-api
exit code: 0
podman rm shopease-db
shopease-db
exit code: 0
sai@ubuntu: ~/shopease-orders$
```

Step 10: Configure & Deploy Staging Pod (podman kube play)

Navigate to kube/ directory and configure Kubernetes Pod manifest kube/order-pod.yaml:

```
sai@ubuntu: ~/shopease-ord$ ls
app  compose.yaml  db  kube  nginx
sai@ubuntu: ~/shopease-ord$ cd kube
sai@ubuntu: ~/shopease-ord/kube$ pwd
/home/sai/shopease-ord/kube
sai@ubuntu: ~/shopease-ord/kube$ |
```

```
sai@ubuntu: ~/shopease-ord$ vim order-pod.yaml
```

Deploy staging pod using podman kube play and verify active Pod status:

podman kube play kube/order-pod.yaml

podman pod ps

podman ps --pod --filter pod=shopease-orders

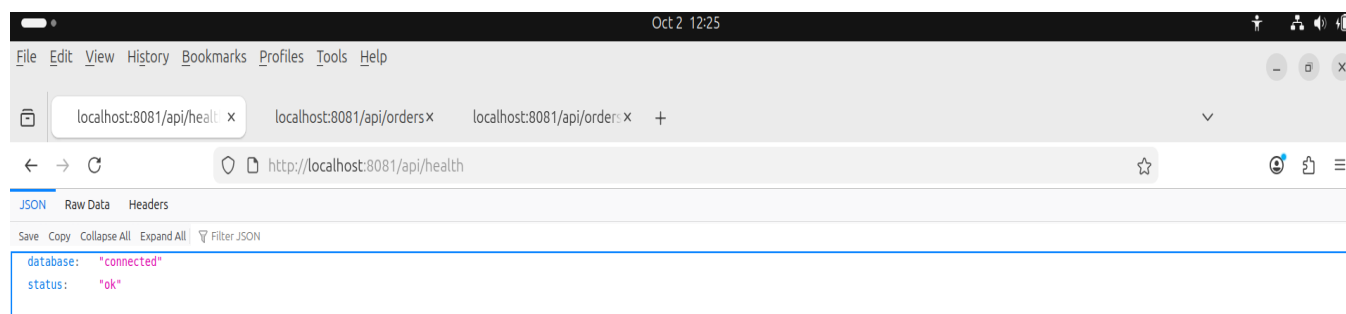
```
sai@ubuntu: ~/shopease-ord$ podman kube play order-pod.yaml
Pod:
8f32bab18a17d141dc18a2e99a0cdd65ed8fd0a3e41d1d655ff268c1172fbbc6
Containers:
a5d1bca4d0ec4912932c20f43427d1fcee86e7411b445aa5717ddb1f76eef67
248bf754b2a8f2c376a953fda87865bb7980118db9497aadb49638f751357fb9
0bd189dbfbcdb581d562bcd1e14db3427183569c58bfe68ee6d9426c91554c9a

sai@ubuntu: ~/shopease-ord/kube$ podman pod ps
POD ID      NAME          STATUS      CREATED      INFRA ID      # OF CONTAINERS
8f32bab18a17 shopease-orders Running     27 seconds ago b2d30098eb76 4
sai@ubuntu: ~/shopease-ord/kube$ podman ps --pod --filter pod=shopease-orders
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
b2d30098eb76   localhost/podman-pause:4.9.3-0      #0                      53 seconds ago Up 50 seconds 0.0.0.0:8081->80/tcp 8f32bab18a17-infra
8f32bab18a17   shopease-orders                      #1                      52 seconds ago Up 50 seconds 0.0.0.0:8081->80/tcp shopease-orders-db
a5d1bca4d0ec   localhost/shopease-order-db:1.0      postgres                52 seconds ago Up 49 seconds 0.0.0.0:8081->80/tcp shopease-orders-api
248bf754b2a8   localhost/shopease-order-api:1.0     gunicorn --bind 0...    52 seconds ago Up 49 seconds 0.0.0.0:8081->80/tcp shopease-orders-gatew
0bd189dbfbcdb localhost/shopease-gateway:1.0        nginx -g daemon o...    52 seconds ago Up 49 seconds 0.0.0.0:8081->80/tcp shopease-orders-gatew
ay 8f32bab18a17 shopease-orders                      #2                      52 seconds ago Up 49 seconds 0.0.0.0:8081->80/tcp shopease-orders-gatew
sai@ubuntu: ~/shopease-ord/kube$ |
```

Step 11: Staging Verification & Direct Database Audit (Port 8081)

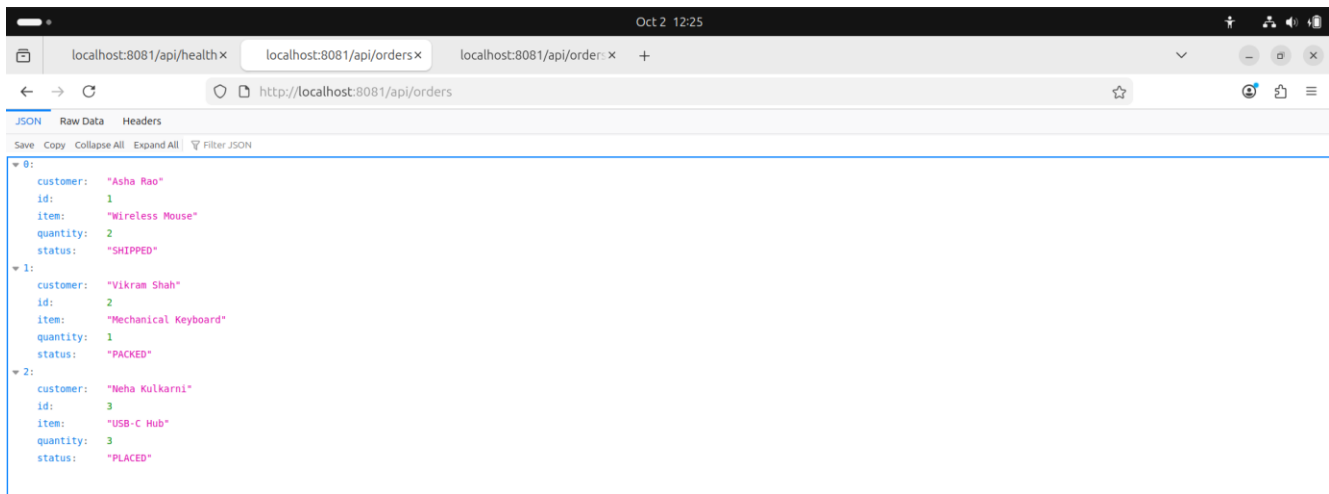
Conduct complete functional verification against staging environment running on <http://localhost:8081>:

curl http://localhost:8081/api/health



```
Oct 2 12:25
File Edit View History Bookmarks Profiles Tools Help
localhost:8081/api/health x localhost:8081/api/orders x localhost:8081/api/order: x +
http://localhost:8081/api/health
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
{
  "database": "connected",
  "status": "ok"
}
```

curl http://localhost:8081/api/orders



podman logs shopease-orders-api

```
sai@ubuntu: ~/shopease-ord$ podman logs shopease-orders-api
[2026-10-02 06:49:00 +0000] [1] [INFO] Starting gunicorn 22.0.0
[2026-10-02 06:49:00 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-10-02 06:49:00 +0000] [1] [INFO] Using worker: sync
[2026-10-02 06:49:00 +0000] [2] [INFO] Booting worker with pid: 2
[2026-10-02 06:49:00 +0000] [3] [INFO] Booting worker with pid: 3
sai@ubuntu: ~/shopease-orders/kube$
```

podman exec -it shopease-orders-db psql -U shopease -d orders -c "SELECT * FROM orders;"

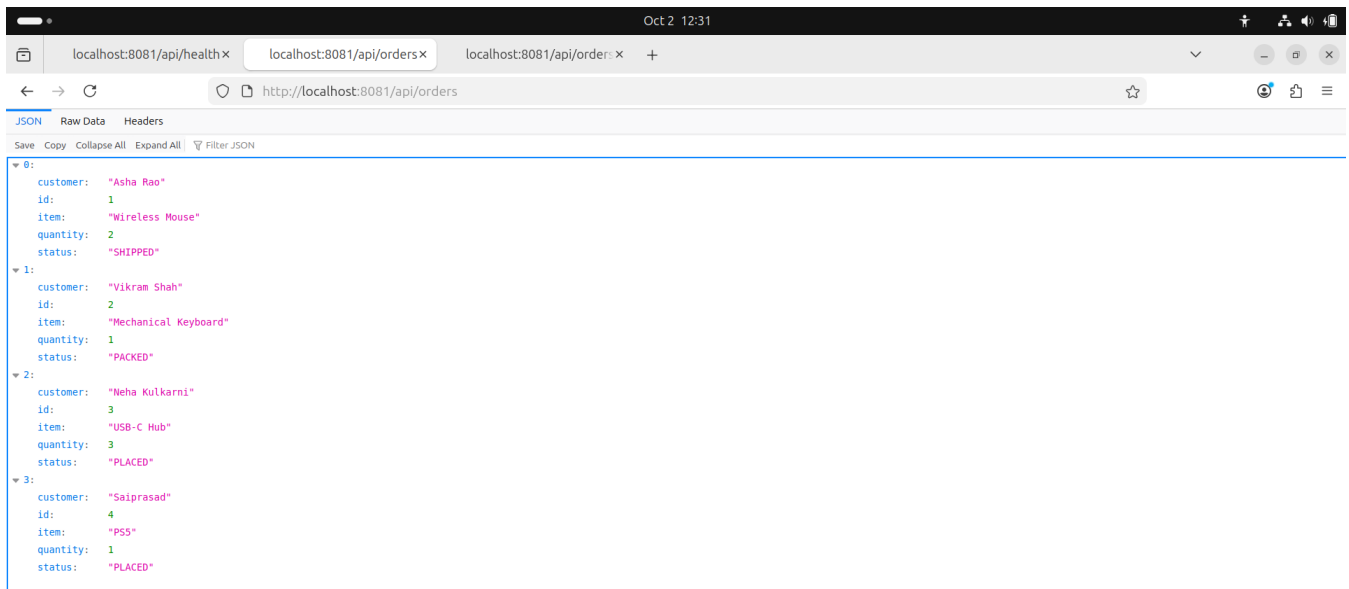
```
sai@ubuntu: ~/shopease-ord$ podman exec -it shopease-orders-db psql -U shopease -d orders -c "SELECT * FROM orders;"
 id | customer | item          | quantity | status | created_at
-----+-----+-----+-----+-----+-----
  1 | Asha Rao | Wireless Mouse |         2 | SHIPPED | 2026-10-02 06:49:11.65431
  2 | Vikram Shah | Mechanical Keyboard |         1 | PACKED | 2026-10-02 06:49:11.65431
  3 | Neha Kulkarni | USB-C Hub |         3 | PLACED | 2026-10-02 06:49:11.65431
(3 rows)

sai@ubuntu: ~/shopease-orders/kube$
```

curl -X POST http://localhost:8081/api/orders -H "Content-Type: application/json" -d '{"customer":"Suresh Kumar","item":"PS5","quantity":1}'

```
sai@ubuntu: ~/shopease-ord$ curl -X POST http://localhost:8081/api/orders -H "Content-Type: application/json" -d '{"customer":"Saiprasad","item":"PS5","quantity":1}'
{"customer":"Saiprasad","id":4,"item":"PS5","quantity":1,"status":"PLACED"}
sai@ubuntu: ~/shopease-orders/kube$ podman exec -it shopease-orders-db psql -U shopease -d orders -c "SELECT * FROM orders;"
 id | customer | item          | quantity | status | created_at
-----+-----+-----+-----+-----+-----
  1 | Asha Rao | Wireless Mouse |         2 | SHIPPED | 2026-10-02 06:49:11.65431
  2 | Vikram Shah | Mechanical Keyboard |         1 | PACKED | 2026-10-02 06:49:11.65431
  3 | Neha Kulkarni | USB-C Hub |         3 | PLACED | 2026-10-02 06:49:11.65431
  4 | Saiprasad | PS5 |         1 | PLACED | 2026-10-02 06:59:35.127084
(4 rows)

sai@ubuntu: ~/shopease-orders/kube$
```



Step 12: Pod Lifecycle, Kubernetes Manifest Generation & Final Teardown

Test staging Pod lifecycle operations, generate Kubernetes YAML from running Pod, and execute final cleanup:

podman kube down kube/order-pod.yaml

```
sai@ubuntu: ~/shopease-ord  X  +  v
sai@ubuntu:~/shopease-orders/kube$ podman kube down order-pod.yaml
Pods stopped:
8f32bab18a17d141dc18a2e99a0cdd65ed8fd0a3e41d1d655ff268c1172fbbc6
Pods removed:
8f32bab18a17d141dc18a2e99a0cdd65ed8fd0a3e41d1d655ff268c1172fbbc6
Secrets removed:
Volumes removed:
sai@ubuntu:~/shopease-orders/kube$ podman pod ps
POD ID      NAME          STATUS    CREATED      INFRA ID    # OF CONTAINERS
sai@ubuntu:~/shopease-orders/kube$
```

podman kube play kube/order-pod.yaml

```
sai@ubuntu:~/shopease-orders/kube$ podman kube play order-pod.yaml
Pod:
d56e51d44aa7418a0caf1d2d07d16a0f76c10c6e879f4527b249115a838dfc8f
Containers:
87dee8aec29ac6c51cf37274272086a4745d58d7463285e21e527b35a48c9d15
e489c65a507a7e1809be275dc48416ded1c900b3db1c1d4673adcf753ccaf716
fcc20d48ddd1cb1cf9aa6697bb249d5c818fb8dea6922b8037cf680ae36319e8
sai@ubuntu:~/shopease-orders/kube$ podman pod ps
POD ID      NAME          STATUS    CREATED      INFRA ID    # OF CONTAINERS
d56e51d44aa7  shopease-orders  Running   8 seconds ago  036bad98cc21  4
```

podman kube generate shopease-orders -s -f generated-pod.yaml

diff kube/order-pod.yaml generated-pod.yaml

```
sai@ubuntu: ~/shopease-orders  X  +  v
sai@ubuntu:~/shopease-orders/kube$ podman kube generate shopease-orders -s -f generated-pod.yaml
sai@ubuntu:~/shopease-orders/kube$ diff order-pod.yaml generated-pod.yaml
0a1,4
> # Save the output of this file and use kubectl create -f to import
> # it into Kubernetes.
> #
> # Created with podman-4.9.3
2,3c6
< kind: Pod
<
---
> kind: Service
4a8,10
>   creationTimestamp: "2026-10-02T07:05:23Z"
>   labels:
>     app: shopease-orders
5a12,25
> spec:
>   ports:
>     - name: "80"
>       nodePort: 32504
>       port: 80
>       targetPort: 80
>   selector:
>     app: shopease-orders
>   type: NodePort
> ---
> apiVersion: v1
> kind: Pod
> metadata:
>   creationTimestamp: "2026-10-02T07:05:23Z"
8,9c28
<   env: staging
<
---
>   name: shopease-orders
11,12d29
<   restartPolicy: OnFailure
```

```
sai@ubuntu: ~/shopease-orders  X  +  v
sai@ubuntu:~/shopease-orders/kube$ podman kube down order-pod.yaml
Pods stopped:
d56e51d44aa7418a0caf1d2d07d16a0f76c10c6e879f4527b249115a838dfc8f
Pods removed:
d56e51d44aa7418a0caf1d2d07d16a0f76c10c6e879f4527b249115a838dfc8f
Secrets removed:
Volumes removed:
sai@ubuntu:~/shopease-orders/kube$ |
```

Key Takeaways:

- ✓ Same Images, Different Environments: We use the same container images for both development and staging. Only the configuration changes.
- ✓ Different Networking: In Podman Compose, containers communicate using service names like api and db. In a Pod, all containers share the same network and communicate through 127.0.0.1.
- ✓ Flexible Configuration: Using environment variables such as DB_HOST and API_HOST allows the same application to work in both environments without changing the application code.
- ✓ Run Kubernetes YAML Locally: podman kube play allows us to run a Kubernetes-style Pod locally using Podman, without needing a Kubernetes cluster.

- ✓ **Generate Kubernetes YAML:** `podman kube generate` can create Kubernetes YAML from a running Pod, which can be useful when moving the workload toward platforms such as OpenShift or GKE.

Conclusion:

This project successfully demonstrated how to build and run the ShopEase Order Service using Podman. We created separate containers for Nginx, the Flask API, and PostgreSQL, and used the same images for both development with Podman Compose and staging with a Podman Pod.

The project also showed important practices such as running the API as a non-root user, using Gunicorn, keeping backend services unexposed to the host, and using environment variables for flexible configuration. We verified the application using API requests, container logs, and PostgreSQL queries, confirming that the complete application was working correctly in both environments.